INTERNATIONAL INSTITUTE OF MEDICAL SCIENCE & TECHNOLOGY COUNCIL

A Mini Project Documentation on

"Urban Air Quality Index Modeler"

Submitted in partial Fulfillment of the requirement for internship on

"AIML and Gen AI"

Submitted by

COHORT- 10

Pooja H

B V Vedamurthi

Kashish

Savari Yevoor

Lasya T P

Narayan M Nayak

Chandushree S

Manisha M S

Dikshitha Ramesh

Priyaguna R

Tejas V

Tamirah Sharieff

Ammar Shibli

Nithin N Patel

Snehal Rai B

Abstract

This project proposes a robust machine learning pipeline designed to forecast PM2.5 concentrations by analyzing a comprehensive dataset of environmental variables. The methodology incorporates key pollutant indicators (PM10, SO_2, NO_2, CO, O_3) alongside vital meteorological factors (Temperature, Pressure, Dew Point, Rainfall, Wind Speed). Furthermore, rigorous data preprocessing and feature engineering were conducted to extract crucial temporal patterns, specifically the Month and Hour of the day, ensuring the model captures both seasonal trends and daily fluctuations. The dataset was subsequently partitioned using an 80:20 train-test split to ensure reliable validation.

To determine the optimal predictive engine, three distinct regression algorithms Linear Regression, Random Forest, and XGBoost were implemented and evaluated using Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and R^2 scores. Comparative analysis revealed that the XGBoost algorithm delivered the best overall performance, providing the highest predictive accuracy while maintaining computational efficiency. Additionally, feature importance analysis highlighted that PM10 and Carbon Monoxide (CO) serve as the most dominant predictors of PM2.5 levels, offering valuable insights into local pollution dynamics.

To demonstrate practical utility, the optimized XGBoost model was integrated into a local Streamlit web application. This interactive interface allows users to input real-time environmental parameters and instantly visualize PM2.5 predictions. Ultimately, this system supports data-driven decision-making and directly aligns with the United Nations Sustainable Development Goal 11 (SDG 11): Sustainable Cities and Communities, by providing a scalable framework to help build safer, more resilient urban environments.

Table of Contents

1. Introduction

2. Problem Statement

3. Methodology

4. System Architecture

5. Model Development

    5.1 Algorithm Selection & Comparison
    5.2 Performance Metrics

6. Implementation

    6.1 Data Preprocessing code
    6.2 Model Development code
    6.3 Deployment code(Streamlit)

7. Results

    7.1 Model Tuning & Performance
    7.2 Model Prediction Visualization
    7.3 Feature Importance Analysis
    7.4 Web Interface Output

8. SDG 11 Impact & Analysis

9. Conclusion

10. References

Introduction

Rapid urbanization, widespread industrialization, and a constant surge in vehicular emissions have collectively precipitated a severe deterioration in global air quality, with fine particulate matter (PM2.5) representing one of the most critical environmental health hazards. Because of their microscopic size, these particles can bypass the body's natural defensive mechanisms, penetrating deep into the respiratory system and even entering the bloodstream, which is directly linked to chronic respiratory conditions and cardiovascular diseases. Traditionally, environmental agencies have relied on physical monitoring stations that are inherently reactive, only alerting the public after hazardous conditions have already materialized. To effectively protect vulnerable populations and implement timely interventions, there is an urgent need to transition from reactive monitoring to proactive forecasting. Addressing this gap, this project leverages advanced machine learning techniques to develop a robust predictive model for PM2.5 concentrations using comprehensive historical datasets that encompass both air quality metrics and localized meteorological factors. The ultimate objective of this study is not only to achieve high predictive accuracy but also to ensure the model is practically accessible. Consequently, the optimized machine learning algorithm is integrated into a localized, interactive web application built with Streamlit. This interface allows end-users to input real-time environmental parameters and instantly visualize forecasted PM2.5 levels, bridging the gap between raw data science and a user-friendly deployment to provide a scalable, data-driven tool that directly supports sustainable urban management.

Problem Statement

Urban air pollution levels fluctuate significantly due to the complex interplay between traffic density, fluctuating weather patterns, and seasonal transitions. While modern monitoring systems are effective at providing real-time data, they often fall short in offering the predictive foresight necessary for proactive public health management. To bridge this gap, there is a critical need for a reliable machine learning based system capable of accurately forecasting PM2.5 concentrations by capturing the intricate, non-linear relationships between various pollutants and atmospheric factors. By integrating these sophisticated modeling techniques into an accessible web platform such a system would empower stakeholders and the general public with actionable, real-time predictions to better navigate environmental risks.

Methodology

1. Data Preprocessing and Analysis: Conduct thorough cleaning and exploratory data analysis (EDA) on air quality datasets to identify key trends and ensure data integrity.

2. Model Development and Comparison: Implement and compare three distinct machine learning algorithms: Linear Regression, Random Forest, and XGBoost to determine the most effective approach for predicting PM2.5 levels.

3. Model Performance Evaluation: Assess the accuracy and reliability of each model using standardized metrics, including Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and R^2 score.

4. Interactive Model Deployment: Develop and implement a real-time prediction interface locally using the Streamlit framework, allowing users to interact with the best performing model.

5. Support for Sustainable Urban Planning: Align the project outcomes with UN Sustainable Development Goal 11 to provide actionable insights for building safer, more resilient, and sustainable cities.

System Architecture

The system architecture for the PM2.5 prediction model follows a comprehensive machine learning pipeline. As illustrated in Figure 1 the workflow progresses from raw data ingestion and preprocessing, through feature engineering and parallel model training, and concludes with the deployment of the optimized XGBoost model via a Streamlit web interface.

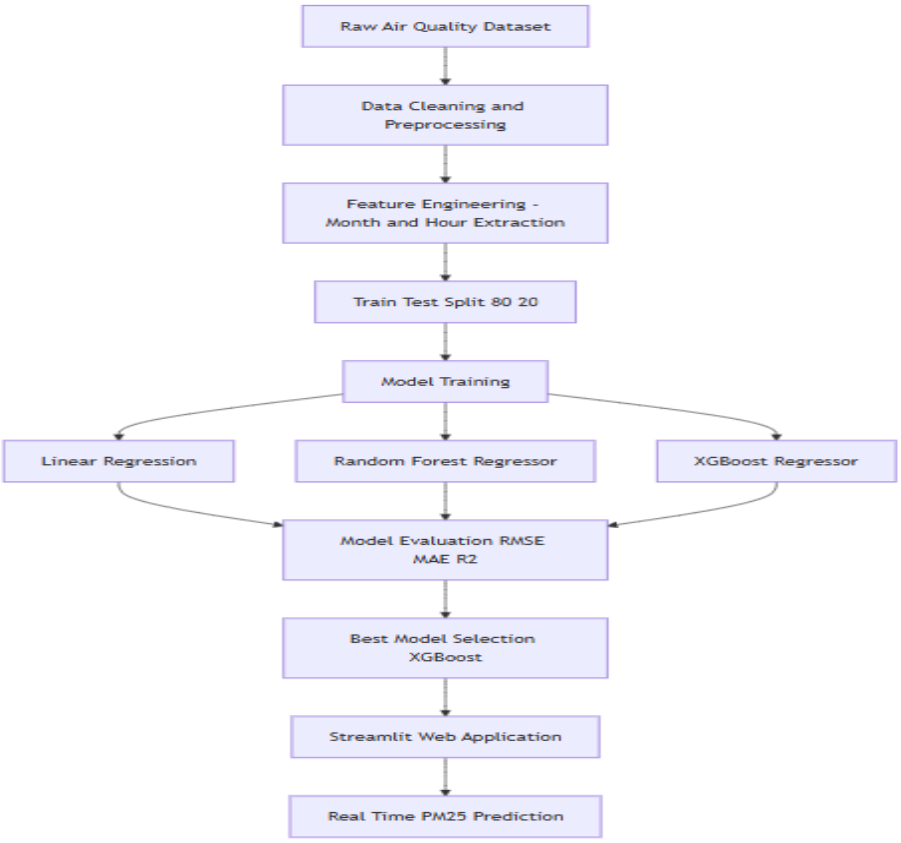


Figure 1: System Architecture of the PM2.5 Prediction Model

Model Development

Algorithm Selection & Comparison

To find the most accurate way to predict PM2.5 levels, we initially trained and evaluated three different machine learning algorithms:

1. Linear Regression: Used as a baseline model to understand basic relationships in the data.
2. Random Forest: Applied to capture non-linear patterns and complex interactions between variables.
3. XGBoost: Tested for its advanced gradient boosting framework, which is highly optimized for tabular data.

Performance Metrics

We compared the models based on their Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and R-squared (R^2) scores:

- Linear Regression: $R^2 = 0.852$ | MAE = 20.03 | RMSE = 29.97
- Random Forest: $R^2 = 0.938$ | MAE = 11.41 | RMSE = 19.43
- XGBoost: $R^2 = 0.957$ | MAE = 9.75 | RMSE = 16.24

As the metrics show, XGBoost clearly outperformed the other two algorithms. It achieved the highest accuracy while keeping the prediction errors to a minimum.

Because it provided the best balance of high performance and reliability, we selected XGBoost as the final model for our real-time prediction system. Additionally, we plotted the model's Feature Importance to visualize exactly which input variables had the strongest impact on the final PM2.5 predictions.

Implementation

Data Preprocessing code:

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import StandardScaler

df = pd.read_csv("Beijing Multisite air Quality data.csv")

df = df.dropna()

df['datetime'] = pd.to_datetime(df[['year','month','day','hour']])

df['Hour'] = df['datetime'].dt.hour

df['wd'] = df['wd'].astype('category').cat.codes

numerical_cols = ['PM2.5','PM10','TEMP','PRES','DEWP','RAIN','WSPM']

for col in numerical_cols:

    Q1 = df[col].quantile(0.25)

    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1
```

```
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df = df[(df[col] >= lower) & (df[col] <= upper)]

features = ['TEMP','PRES','DEWP','RAIN','wd','WSPM','Hour']
X = df[features]
y = df['PM2.5']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=features)

# Export cleaned data
df.to_csv("final_air_quality_dataset.csv", index=False)
```

Model Development code:

```
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

Urban Air Quality Index Modeler

```
data = pd.read_csv("https://raw.githubusercontent.com/PRIYAGUNAR/IIMSTC-Cohort10-Urban-AQI-Modeler/main/cleaned_air_quality_data%20(1)%20(1).csv")


# Define Target & Features
X = data.select_dtypes(include=['number', 'bool']).drop(columns=['PM2.5'])
y = data['PM2.5']


# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Evaluation Function
def evaluate(y_true, y_pred, model_name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    print(f"{model_name} -> RMSE: {rmse}, MAE: {mae}, R2: {r2}")


# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)


# Random Forest
rf = RandomForestRegressor(
    n_estimators=300,
    max_depth=20,
    min_samples_leaf=5,
```

```
    n_jobs=-1,
    random_state=42)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)

# XGBoost
xgb = XGBRegressor(
    n_estimators=500,
    learning_rate=0.03,
    max_depth=8,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1.0,
    random_state=42)
xgb.fit(X_train, y_train)
pred_xgb = xgb.predict(X_test)

evaluate(y_test, pred_lr, "Linear Regression")
evaluate(y_test, pred_rf, "Random Forest")
evaluate(y_test, pred_xgb, "XGBoost")

joblib.dump(xgb, "pm25_xgb_model.pkl")
joblib.dump(rf, "pm25_rf_model.pkl")
```

Deployment Code (Streamlit):

```
import streamlit as st
```

Urban Air Quality Index Modeler

```
import joblib
import numpy as np
import os

st.set_page_config(
    page_title="PM2.5 Air Quality Predictor",
    layout="wide"
)
st.title("PM2.5 Air Quality Predictor")
st.markdown("Predict PM2.5 concentration levels based on air quality parameters")

@st.cache_resource
def load_model():
    # Load the XGBoost model from the repo `models/` folder (tracked in git)
    repo_root = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
    model_path = os.path.join(repo_root, "models", "pm25_xgb_model.pkl")

    if not os.path.exists(model_path):
        raise FileNotFoundError(
            f"Model file not found at: {model_path}. "
            "Make sure `models/pm25_xgb_model.pkl` exists in the deployed branch."
        )
    return joblib.load(model_path)

try:
    model = load_model()
    st.success("✓ Model loaded successfully")
```

Urban Air Quality Index Modeler

```
except Exception as e:
    st.error(f"Error loading model: {e}")
    st.stop()


st.sidebar.header("Input Parameters")
st.sidebar.markdown("---")
col1, col2 = st.sidebar.columns(2)


with col1:
    pm10 = st.sidebar.number_input("PM10 (µg/m³)", min_value=2.0, max_value=992.0, value=104.0, step=1.0, format="%.1f")
    so2 = st.sidebar.number_input("SO2 (ppb)", min_value=0.57, max_value=310.0, value=17.0, step=0.5, format="%.2f")
    no2 = st.sidebar.number_input("NO2 (ppb)", min_value=2.0, max_value=290.0, value=53.0, step=1.0, format="%.1f")
    co = st.sidebar.number_input("CO (ppm)", min_value=100.0, max_value=10000.0, value=1211.0, step=10.0, format="%.0f")
    o3 = st.sidebar.number_input("O3 (ppb)", min_value=0.21, max_value=429.0, value=58.0, step=1.0, format="%.2f")
    temp = st.sidebar.number_input("Temperature (°C)", min_value=-16.8, max_value=41.4, value=14.0, step=0.5, format="%.1f")


with col2:
    pres = st.sidebar.number_input("Pressure (hPa)", min_value=982.4, max_value=1042.0, value=1010.0, step=0.5, format="%.1f")
    dewp = st.sidebar.number_input("Dew Point (°C)", min_value=-35.3, max_value=28.5, value=2.65, step=0.5, format="%.2f")
    rain = st.sidebar.number_input("Rainfall (mm)", min_value=0.0, max_value=52.1, value=0.1, step=0.5, format="%.1f")
```

Urban Air Quality Index Modeler

```
    wspm = st.sidebar.number_input("Wind Speed (m/s)", min_value=0.0, max_value=11.2,
value=1.8, step=0.1, format="%.1f")

    month = st.sidebar.selectbox("Month", list(range(1,13)), index=5)

    hour = st.sidebar.number_input("Hour (0-23)", min_value=0, max_value=23, value=12,
step=1, format="%d")


features = np.array([[pm10, so2, no2, co, o3, temp, pres, dewp, rain, wspm, month, hour]])

raw_prediction = float(model.predict(features)[0])

prediction = max(0.0, raw_prediction)

st.markdown("---")

col_metric1, col_metric2 = st.columns([2, 1])


with col_metric1:

    st.metric(

        label="Predicted PM2.5 Concentration",

        value=f"{prediction:.2f} µg/m³",

        delta=None,

        delta_color="normal"

    )


def get_aqi_status(pm25):

    if pm25 <= 35:

        status = "GOOD"

        color = "#00b050"

        advice = "Air quality is satisfactory. Enjoy outdoor activities!"

    elif pm25 <= 75:

        status = "MODERATE"
```

```
        color = "#ffc000"
        advice = "Air quality is acceptable. Sensitive groups may experience issues."
    elif pm25 <= 115:
        status = "UNHEALTHY FOR SENSITIVE GROUPS"
        color = "#ff6600"
        advice = "Sensitive groups should limit outdoor activities."
    elif pm25 <= 150:
        status = "UNHEALTHY"
        color = "#ff0000"
        advice = "Everyone may begin to experience health effects. Limit outdoor activities."
    else:
        status = "VERY UNHEALTHY"
        color = "#8b0000"
        advice = "Health alert! Avoid outdoor activities."

    return status, color, advice

status, color, advice = get_aqi_status(prediction)

with col_metric2:
    st.markdown(f"<h3 style='color: {color}'>{status}</h3>", unsafe_allow_html=True)

st.info(f"**Advisory:** {advice}")

st.markdown("---")
st.subheader("Input Summary")
```

Urban Air Quality Index Modeler

```
input_data = {
    "Parameter": [
        "PM10", "SO2", "NO2", "CO", "O3",
        "Temperature", "Pressure", "Dew Point", "Rainfall", "Wind Speed",
        "Month", "Hour"
    ],
    "Value": [
        pm10, so2, no2, co, o3,
        temp, pres, dewp, rain, wspm,
        month, hour
    ],
    "Unit": [
        "µg/m³", "ppb", "ppb", "ppm", "ppb",
        "°C", "hPa", "°C", "mm", "m/s",
        "", ""
    ]
}

col1, col2, col3 = st.columns(3)

with col1:
    st.write("**Pollutants**")
    st.write(f"PM10: {pm10} µg/m³")
    st.write(f"SO2: {so2} ppb")
    st.write(f"NO2: {no2} ppb")
    st.write(f"CO: {co} ppm")
    st.write(f"O3: {o3} ppb")
```

```
with col2:
    st.write("**Weather**")
    st.write(f"Temperature: {temp}°C")
    st.write(f"Pressure: {pres} hPa")
    st.write(f"Dew Point: {dewp}°C")
    st.write(f"Rainfall: {rain} mm")
    st.write(f"Wind Speed: {wspm} m/s")

with col3:
    st.write("**Time**")
    months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov",
"Dec"]
    st.write(f"Month: {months[month-1]}")
    st.write(f"Hour: {hour:02d}:00")

st.markdown("---")
st.markdown("---")
st.markdown("""
<div style='text-align: center'>
    <p style='font-size: 12px; color: gray;'>
    IIMSTC Cohort 10 - Urban Air Quality Index Modeler
    </p>
</div>
""", unsafe_allow_html=True)
```

Results

1. Model Tuning & Performance

Three algorithms were evaluated to find the best predictive model for PM2.5 concentrations. As shown in Table 1, XGBoost significantly outperformed the others. It achieved the highest accuracy, successfully explaining nearly 96% of the data's variance ($R^2 = 0.957$), while minimizing prediction errors to the lowest levels (RMSE = 16.24, MAE = 9.75). Due to this superior accuracy and reliability compared to Random Forest and Linear Regression, XGBoost was selected as the final model for deployment.

Model	RMSE ↓	MAE ↓	R^2 ↑	Overall Performance
Linear Regression	29.97	20.03	0.852	Baseline (Lowest Accuracy)
Random Forest	19.43	11.41	0.938	Good
XGBoost	16.24	9.75	0.957	Best Performance

Table 1: Final Performance Metrics

Model comparison

```
In [16]: evaluate(y_test, pred_lr, "Linear Regression")

Linear Regression
RMSE: 29.97338692638447
MAE : 20.03053345551795
R2  : 0.85198757241538

In [17]: evaluate(y_test, pred_rf, "Random Forest")

Random Forest
RMSE: 19.42620792417022
MAE : 11.409833772116587
R2  : 0.9378268878310767

In [18]: evaluate(y_test, pred_xgb, "XGBoost")

XGBoost
RMSE: 16.238425117346893
MAE : 9.747130594383238
R2  : 0.9565575454500709
```

Figure 2. Model Performance

2. Model Prediction Visualization

As shown in Figure 3, the actual versus predicted PM2.5 concentrations tightly cluster along the perfect prediction line. This strong visual alignment confirms the XGBoost model's high R^2 score of 0.957 and its minimal error.

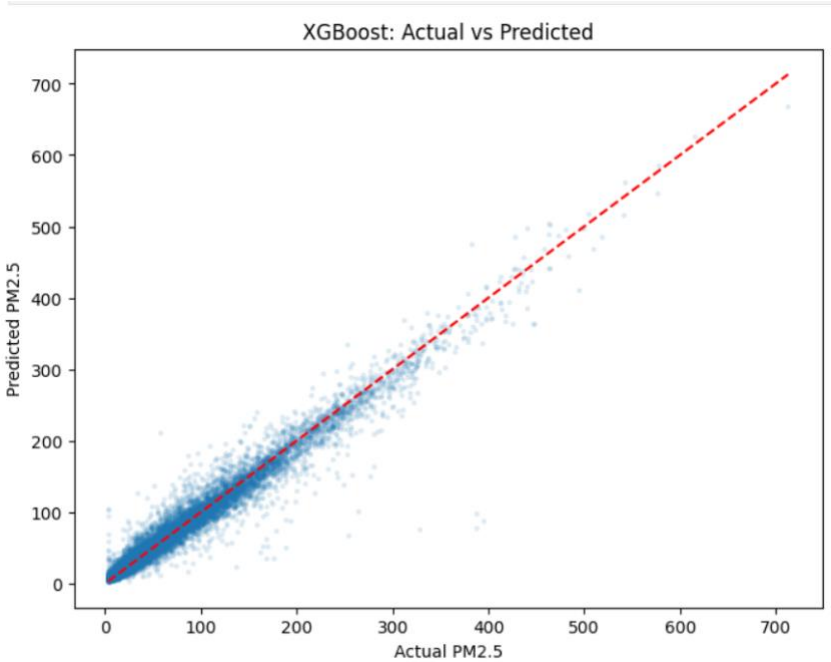


Figure 3. Scatter plot of Actual vs. Predicted PM2.5 concentrations for the optimized XGBoost model.

3. Feature Importance Analysis

To interpret the XGBoost model's predictions, feature importance scores were extracted. As illustrated in Figure 4 below, PM10 is the dominant predictor for PM2.5, followed by CO. Meteorological factors and engineered temporal features (month/hour) had minimal impact on the model's decision-making process.

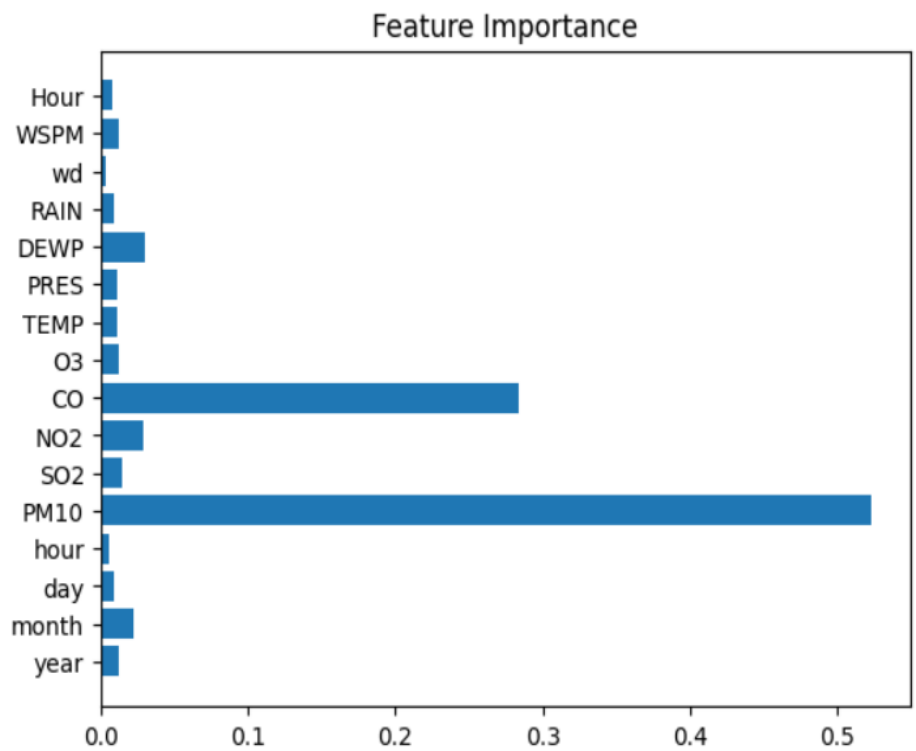
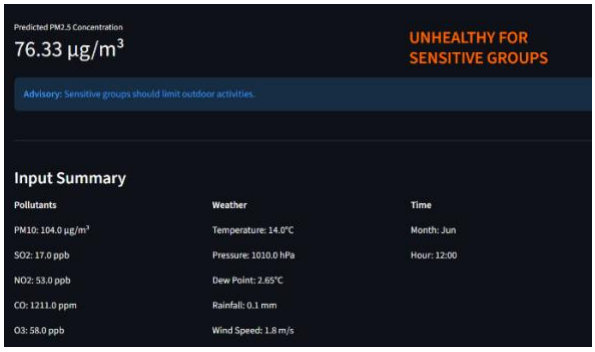
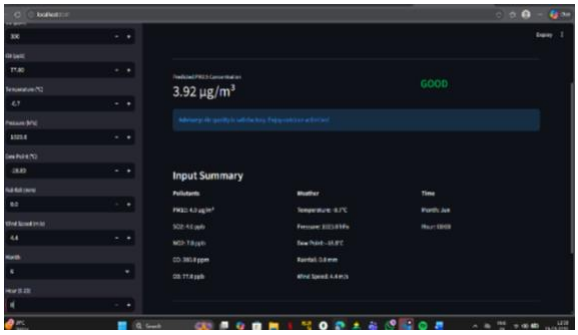


Figure 4: Feature importance scores for the optimized XGBoost model

4. Real-Time Prediction Output

Urban Air Quality Index Modeler





SDG 11 Impact & Analysis

This project directly supports the United Nations Sustainable Development Goal 11 (SDG 11), which aims to make cities and human settlements inclusive, safe, resilient, and sustainable. Specifically, the model aligns with Target 11.6, which emphasizes reducing the adverse per capita environmental impact of cities by paying special attention to air quality.

The deployment of this real-time PM2.5 prediction system provides actionable value in the following ways:

- **Proactive Public Health Protection:** By accurately forecasting PM2.5 spikes using the XGBoost model, the system serves as an early warning tool. This allows municipalities to issue timely health advisories, enabling vulnerable populations to take precautions and reducing respiratory-related hospitalizations.
- **Data-Driven Urban Policy**: The feature importance analysis revealed that PM10 and CO are the dominant drivers of PM2.5 pollution. This precise insight empowers city planners and environmental agencies to move away from guesswork and target specific emission sources (like vehicular exhaust or industrial CO output) with strict, localized regulations.

- **Resource Optimization**: Instead of reacting to pollution after it has blanketed a city, local governments can use these real-time predictions to implement temporary preventive measures such as rerouting traffic or pausing specific industrial activities only when the predictive model indicates a critical threshold will be breached**.**

Conclusion

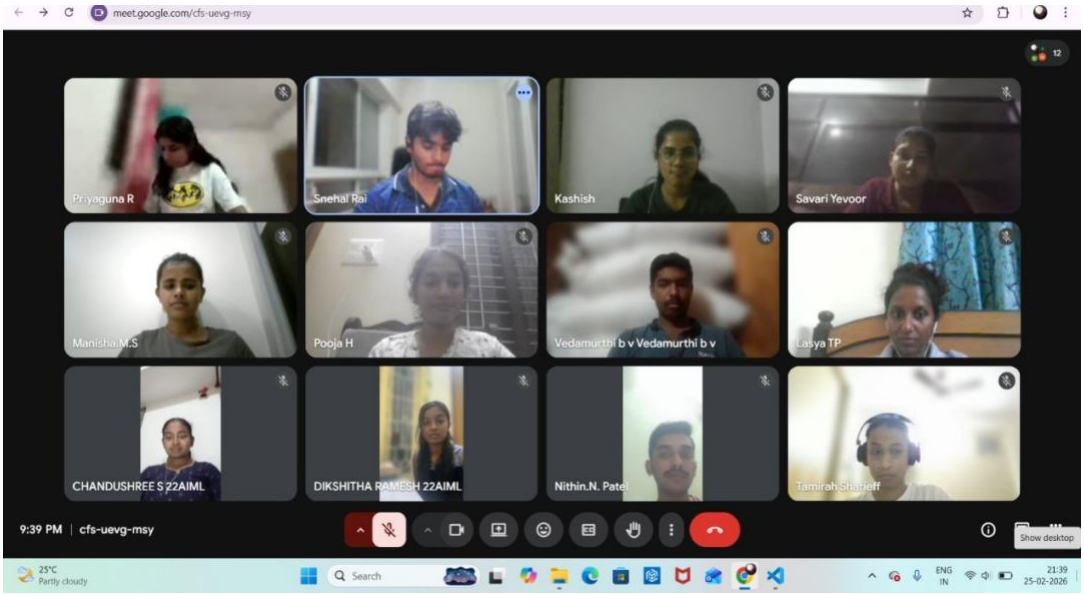
This study successfully developed a highly reliable machine learning pipeline for predicting PM2.5 air pollution levels. After rigorously evaluating Linear Regression, Random Forest, and XGBoost algorithms, XGBoost clearly emerged as the optimal solution. It demonstrated superior predictive reliability, successfully explaining approximately 96% of the variance in PM2.5 concentrations while maintaining the lowest error margins. Furthermore, the feature importance analysis validated the model's scientific soundness by correctly identifying PM10 and Carbon Monoxide (CO) as the primary drivers of PM2.5 fluctuations. By successfully integrating this optimized model into a local Streamlit web interface, this project proves that advanced machine learning can be effectively translated into a practical, real-time tool to monitor air quality and support sustainable urban environments.

References

Data Source: https://www.kaggle.com/datasets/aravindpcoder/beijing-multi-site-air-quality-data

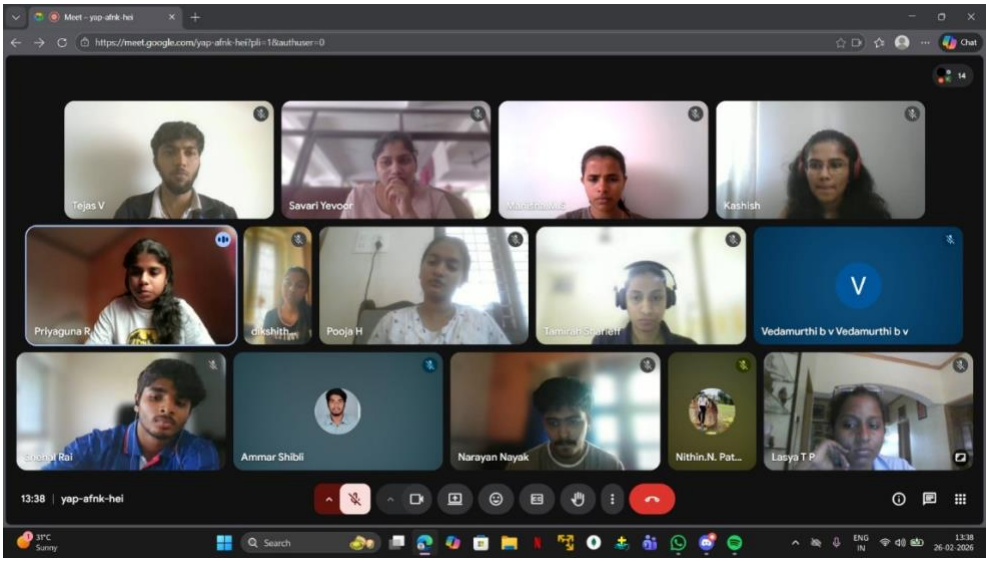
Meeting Documentation:

First Meeting on:25/02/2026



Urban Air Quality Index Modeler

Second meeting on: 26/02/2026



Third Meeting on: 27/02/2026

Urban Air Quality Index Modeler

Fourth meeting on: 28/02/2026

Fifth meeting on: 02/03/2026